

Міністерство освіти і науки України
ВСП «Одеський фаховий коледж комп'ютерних технологій
Одеського національного університету імені І. І. Мечникова»

МЕТОДИЧНІ ВКАЗІВКИ
ДО ВИКОНАННЯ КУРСОВОЇ РОБОТИ

КОМП'ЮТЕРНА СХЕМОТЕХНІКА ТА АРХІТЕКТУРА
КОМП'ЮТЕРА

Галузь знань: **123 «Інформаційні технології»**

спеціальності: 123 «Інженерія апаратно-програмних засобів»

Освітньо-професійна програма: «Інженерія апаратно-програмних засобів»

Одеса – 2026 рік

Розробники:

Ключник Роман Євгенович – викладач комп'ютерних дисциплін спеціаліст
без категорій

(вказати авторів, їхні посади, звання, категорії)

Обговорено та схвалено на засіданні циклової комісії

Програмної інженерії та комп'ютерних систем та мереж

(назва циклової комісії)

Протокол № ____ від « ____ » _____ 2026 року

Голова циклової комісії _____ Максим СМОРЖ

ЗМІСТ

РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ	6
1.1. Функціональна роль тракту даних у структурі ЕОМ	6
1.2. Обґрунтування вибору архітектурних рішень	7
РОЗДІЛ 2. ПРОЄКТУВАННЯ ВУЗЛА ПРОЦЕСОРА	8
2.1. Компонентна база та логіка обробки даних	8
2.2. Розробка структурної схеми та мікроалгоритмів	10
2.3. Розробка структурної схеми взаємодії	12
2.4. Алгоритми виконання базових мікрооперацій	14
РОЗДІЛ 3. ЛОГІЧНЕ МОДЕЛЮВАННЯ ТА СИНТЕЗ	16
3.1. Синтез логічної моделі та верифікація	17
3.2. Верифікація логічної структури	19
РОЗДІЛ 4. АНАЛІЗ РЕЗУЛЬТАТІВ ДОСЛІДЖЕННЯ.....	20
4.1. Часовий аналіз та оцінка продуктивності	21
4.2. Розрахунок критичного шляху та оцінка продуктивності ...	22
ВИСНОВКИ ..	23
ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ	24

КАЛЕНДАРНИЙ ПЛАН

<i>Назва етапів підготовки курсового проєкту</i>	<i>Строк виконання етапів проєкту (№ тижня)</i>
Отримання теми курсової роботи та уточнення об'єкта дослідження	25 березня
Підбір і опрацювання джерел, формування теоретичної бази	30 березня
Аналіз предметної області та постановка задачі	4 квітня
Розробка структурної схеми та логічних/архітектурних моделей	12 квітня
Моделювання та верифікація роботи вузла	20 квітня
Аналіз результатів, підготовка висновків і рекомендацій	30 квітня
Оформлення пояснювальної записки	1 травня
Здача курсової роботи на перевірку	10 травня
Захист курсової роботи	12 травня

Здобувач освіти _____

Керівник проєкту _____

Даніїл Нарижняк

Ключник Роман

ВСТУП

Сьогодні архітектура **RISC-V** — це один із найголовніших трендів у мікропроцесорній техніці. Оскільки вона повністю відкрита та безкоштовна, її активно використовують як у великих технологічних компаніях, так і в університетах для навчання. Щоб розібратися, як працює будь-який сучасний процесор, найкраще почати з його "серця" — тракту даних.

Мета цієї роботи — спроектувати спрощений однотактний тракт даних. Він має виконувати базові команди: додавати, віднімати, логічні операції та переходи. Це дозволить наочно побачити, як взаємодіють між собою блоки всередині процесора.

- **Об'єкт дослідження:** базова 32-бітна архітектура **RV32I**.
- **Предмет дослідження:** як саме дані рухаються та керуються всередині процесора за один такт.

Основні завдання роботи:

1. Розібратися зі структурою ЕОМ.
2. Розробити логічні схеми головних вузлів: арифметико-логічного пристрою (АЛП) та регістрового файлу.
3. Провести часовий аналіз, щоб дізнатися максимальну швидкість роботи (частоту) процесора.

Отримана модель — це базовий фундамент. Розібравшись із нею, у майбутньому можна буде легко переходити до проектування складніших, наприклад, конвеєрних процесорів.

1.1 Функціональна роль тракту даних у структурі ЕОМ

Тракт даних (Datapath) — це сукупність функціональних вузлів процесора, які безпосередньо виконують операції над даними. До них належать арифметико-логічні пристрої, регістри, мультиплексори та шини даних. В архітектурі фон Неймана тракт даних є серцем обчислювального ядра, забезпечуючи перетворення інформації згідно з командами, що надходять з пам'яті.

Взаємодія вузлів побудована на циклі виконання команди, який у спрощеному вигляді включає:

1. **Вибірка інструкції (Fetch):** Використовуючи адресу з програмного лічильника (PC), процесор зчитує 32-бітне слово з пам'яті інструкцій.
2. **Декодування та зчитування регістрів (Decode):** Розбір бітових полів команди та отримання значень з регістрового файлу.
3. **Виконання (Execute):** АЛП проводить обчислення (додавання, віднімання, логічне І тощо).
4. **Доступ до пам'яті (Memory Access):** У командах завантаження або збереження відбувається взаємодія з оперативною пам'яттю.
5. **Зворотний запис (Write Back):** Запис результату обчислень назад у регістровий файл.

У даній роботі розглядається однотактна реалізація, де весь цикл виконується за один період тактового сигналу. Це спрощує логіку керування, але вимагає, щоб тривалість такту була достатньо великою для завершення найдовшої операції.

1.2 Обґрунтування вибору архітектурних рішень

Для проєктування обрано архітектуру RISC-V (RV32I). Головною перевагою є фіксована довжина інструкцій (32 біти), що спрощує логіку декодування та забезпечує високу регулярність тракту даних.

В основу розробки покладено базовий набір команд, наведений у таблиці 1.1.

Таблиця Основні типи інструкцій RV32I

Тип	Призначення	Приклад
R-type	Регістр-регістр операції	ADD, SUB, XOR
I-type	Операції з негайними значеннями / завантаження	ADDI, LW
S-type	Збереження даних у пам'ять	SWX
B-type	Умовні переходи	BEQ, BNE

Вибір 32-бітної розрядності є оптимальним: він забезпечує баланс між складністю схемотехніки та функціональністю. При проєктуванні особлива увага приділяється **критичному шляху** (команда LW), оскільки вона задіює всі основні вузли: вибірку, зчитування регістрів,

Ключові принципи реалізації:

- Гарвардська архітектура: розділення пам'яті команд та даних для уникнення конфліктів доступу в межах одного такту.
- Мінімізація затримок: оптимізація кількості мультиплексорів у ланцюгах передачі операндів.
- Модульність: кожен вузол (АЛП, РС, блок розширення знаку)

2.1 Компонентна база спрощеного процесора

Функціонування тракту даних базується на взаємодії трьох ключових модулів, кожен з яких виконує специфічну роль у процесі обробки інформації:

1. Програмний лічильник (Program Counter, PC):

Спеціалізований 32-бітний регістр, що визначає послідовність виконання програми. На початку кожного такту він видає адресу поточної команди на шину пам'яті. Логіка оновлення \$PC\$ передбачає два сценарії: лінійне виконання (додавання константи 4) або розгалуження (завантаження адреси переходу з виходу АЛП або суматора).

2. Регістровий файл (Register File):

Масив з 32 регістрів загального призначення (\$x0–x31\$). Архітектурною особливістю є те, що регістр \$x0\$ жорстко заземлений (завжди містить 0), що спрощує реалізацію багатьох команд. Файл є трипортовим: два незалежних порти зчитування забезпечують миттєвий доступ до двох операндів, а окремий порт запису дозволяє зберігати результат обчислень у кінці такту.

3. Арифметико-логічний пристрій (ALU):

Центральний вузол обробки, що реалізує набір базових операцій (ADD, SUB, AND, OR, SLT). Окрім основного 32-бітного виходу результату, АЛП формує прапор Zero. Цей сигнал є критичним для виконання команд типу B-type (наприклад, BEQ), оскільки він дозволяє пристрою керування прийняти рішення про виконання переходу.

Для забезпечення гнучкості тракту використовуються мультиплексори

(MUX). Вони виконують роль комутаторів, що обирають джерело даних для АЛП (другий регістр або негайне значення з команди) та джерело для запису в регістровий файл (вихід АЛП або дані з оперативної пам'яті)

2.2. Логіка обробки констант та формування адрес

У архітектурі RISC-V операнди-константи (негайні значення) інтегровані безпосередньо в код інструкції. Через прагнення до регулярності кодування, біти цих значень займають різні позиції залежно від типу команди. Для їх уніфікації у тракці даних використовується блок ImmGen (Immediate Generator).

Функції блоку ImmGen:

- Екстракція та перестановка: Вилучення бітів константи з фіксованих позицій 32-бітного коду команди.
- Розширення знаку (Sign-extension): Перетворення коротких значень (наприклад, 12-бітних) у повноцінне 32-бітне число зі збереженням знаку.
- Формування зміщень: Для команд розгалуження (тип B) та стрибків (тип J) блок ImmGen автоматично враховує зсув бітів, оскільки в RISC-V адреси завжди кратні 2 байтам.

Принципи формування адрес:

Обчислення наступної адреси інструкції в програмному лічильнику (PC) залежить від режиму виконання:

1. Лінійне виконання: $PC = PC + 4$ (стандартний крок для 32-бітних команд).
2. Умовні та безумовні переходи: $PC = PC + ImmGen_offset$ (адресація відносно поточного PC).

Технічна реалізація:

Блок ImmGen реалізований як суто комбінаційна схема (масив мультиплексорів та жорстких з'єднань). Це гарантує мінімальну затримку: як тільки команда потрапляє в регістр інструкцій, сформована константа стає доступною для АЛП або суматора РС без очікування наступного такту

2.3. Розробка структурної схеми взаємодії

Структурна схема об'єднує функціональні вузли в єдину систему, де маршрутизація потоків даних забезпечується набором мультиплексорів (MUX). Координацію роботи цих вузлів здійснює блок керування (Control Unit).

Ключові вузли маршрутизації:

- Селектор операнда АЛП (ALUSrc): Мультиплексор на вході «В» АЛП вибирає між значенням з регістрового файлу та негайним значенням з блоку ImmGen. Це забезпечує підтримку як реєстрових інструкцій (наприклад, add), так і інструкцій з константами (наприклад, addi).
- Селектор запису в регістр (MemtoReg): Визначає джерело даних для запису в регістровий файл. Результат може надходити безпосередньо з виходу АЛП (арифметичні операції) або з пам'яті даних (команди завантаження load).

Шляхи керування (Control Path):

Блок керування працює паралельно з трактом даних. Він декодує код операції (Opcode)

1. RegWrite: дозвіл на оновлення даних у регістровому файлі.
2. ALUSrc: перемикання джерела другого операнда АЛП.
3. MemRead / MemWrite: активація портів читання або запису пам'яті

4. Branch: сигнал, що у поєднанні з результатом АЛП (прапорець Zero) дозволяє виконання умовного переходу.
5. ALUOp: специфікація конкретної операції для АЛП (додавання, віднімання, логічне І тощо).

Така організація дозволяє реалізувати виконання інструкції за один такт (Single-cycle datapath), де кожен керуючий сигнал стабілізується залежно від поточної команди в регістрі інструкцій.

2.4. Алгоритми виконання базових мікрооперацій

Для розуміння роботи спрощеного тракту даних необхідно простежити шлях сигналу та стан керуючих ліній для основних типів інструкцій. Кожна команда проходить через етапи вибірки, декодування, виконання та запису результату.

1. Команда арифметичної обробки: ADD (R-type)

Використовується для операцій над значеннями, що вже знаходяться в регістрах.

- Вибірка: РС передає адресу до пам'яті інструкцій, яка повертає код команди ADD.
- Декодування: Регістровий файл зчитує операнди з адрес rs1 та rs2.
- Налаштування АЛП: Керуючий сигнал $ALUSrc = 0$ спрямовує значення другого регістра на вхід АЛП.
- Виконання: АЛП виконує операцію додавання згідно з кодом $ALUOp$.
- Запис: Сигнал $RegWrite = 1$ дозволяє зберегти результат з виходу АЛП у цільовий регістр rd.

2. Команда завантаження з пам'яті: LW (I-type)

Служить для перенесення даних з оперативної пам'яті в регістровий файл.

- Формування адреси: $ALUSrc = 1$ подає на вхід АЛП значення бази з rs1 та негайне значення (offset) з блоку ImmGen.

- Обчислення: АЛП розраховує ефективну адресу пам'яті як суму $base + offset$.
- Зчитування: Активується сигнал $MemRead = 1$, і дані з RAM надходять у регістр даних пам'яті (MDR).
- Запис: Сигнал $MemtoReg = 1$ перемикає мультиплексор, спрямовуючи дані з пам'яті (а не з АЛП) на вхід запису регістрового файлу. 10

3. Команда умовного переходу: BEQ (B-type)

Змінює послідовність виконання програми, якщо значення двох регістрів рівні.

- **Порівняння:** АЛП виконує віднімання операндів rs1 - rs2.
- **Логіка переходу:** Якщо результат операції дорівнює нулю, АЛП піднімає прапорець **Zero**.
- **Оновлення PC:** Логічна схема "I" (AND) аналізує сигнали Branch (від блоку керування) та Zero. Якщо обидва сигнали активні, програмний лічильник замість стандартного кроку +4 приймає значення PC + ImmGen_offset.
- **Запис:** Для цього типу команд сигнали RegWrite та MemWrite залишаються неактивними (0).

Резюме станів керуючих сигналів:

Команда	RegWrite	ALUSrc	MemRead	MemWrite	MemtoReg	Branch
ADD	1	0	0	0	0	0
LW	1	1	1	0	1	0
BEQ	0	0	0	0	X	0

(Примітка: X — стан, що не має значення для поточної операції)

3.1. Методика генерації та синтезу логічної моделі

Логічне моделювання тракту даних базується на декомпозиції архітектури до рівня елементарних логічних виразів та їх наступному синтезі у фізичну схему. Цей процес дозволяє перевірити коректність роботи процесора ще до етапу фізичного виготовлення.

1. Синтез блоку керування (Control Unit) Блок керування є центральним вузлом синтезу, що визначає поведінку всього тракту. Він проектується як комбінаційна мережа, що реалізує таблицю істинності (Truth Table).

- Вхідні дані: 7-бітний код операції (Opcode), отриманий з регістру.
- Вихідні дані: набір керуючих сигналів (RegWrite, ALUSrc, MemRead тощо). Оптимізація цієї мережі зазвичай виконується методами мінімізації логічних функцій (наприклад, картами Карно або алгоритмом Куайна — Мак-Класкі).

2. Ієрархічний синтез АЛП та декодера

- ALU Decoder: Невеликий логічний блок, який аналізує додаткові поля інструкції (funct3 та funct7) разом із сигналом ALUOp. Це дозволяє використовувати один і той самий Opcode для різних типів операцій (наприклад, розрізняти додавання та віднімання в R-типі).
- Виконавча схема АЛП: Базується на використанні каскаду повних суматорів для арифметики та паралельних логічних гейтів для побітових операцій (OR, AND, XOR).

3. Використання засобів автоматизації (CAD) Для перетворення

високорівневого опису (наприклад, на мовах Verilog або VHDL) у фізичну структуру використовуються системи автоматизованого проектування.

Процес синтезу включає:

Трансляція: Перетворення коду у булеві рівняння.

Генерація нетліста (Netlist): Формування фінальної мережі логічних вентилів, оптимізованої за параметрами площі кристала та критичного шляху затримки сигналу.

3.2. Верифікація логічної структури

Метою верифікації є підтвердження коректності маршрутизації даних у тракті (Dataflow verification) та дотримання часових параметрів. Для перевірки використовуються тестові вектори — набори бінарних сигналів, що імітують реальні інструкції.

Процес тестування охоплює ключові етапи:

1. Ініціалізація (Reset): перевірка встановлення програмного лічильника (PC) у початкову адресу після сигналу скидання.
2. Лінійне виконання: контроль інкременту PC на 4 після кожного циклу вибірки команди.
3. Стрес-тест АЛП: перевірка операцій із граничними значеннями (додавання нуля, операнди з максимальною розрядністю, виникнення переповнення).
4. Валідація розгалужень: тестування команд переходів (наприклад, BEQ) в обох станах умови — коли вона істинна (перехід виконується) та хибна (програма йде далі лінійно).

Основним інструментом аналізу є часові діаграми (Waveforms). Вони візуалізують динаміку перемикання сигналів на кожній лінії шини, дозволяючи виявити затримки, «гонки» сигналів або помилки в логіці керування на конкретних тактах роботи процесора.

4.1 Часовий аналіз та затримки сигналів

Часовий аналіз є критичним для оцінки фізичної реалізованості проєкту. Кожен елемент схеми має власну затримку розповсюдження (propagation delay). Загальний час виконання команди в однотактному процесорі дорівнює сумі затримок усіх блоків у найдовшому ланцюгу.

Основні компоненти затримок:

- t_{pc} : час відгуку лічильника команд.
- t_{mem} : час доступу до пам'яті інструкцій/даних.
- t_{reg} : час зчитування/запису в регістровий файл.
- t_{alu} : час стабілізації результату в АЛП.
- t_{mux} : затримка на мультиплексорах.

Для успішного функціонування сигнал повинен "встоятися" (settle) до моменту наступного фронту тактового імпульсу. Якщо частота занадто висока, дані запишуться некоректно.

4.2. Розрахунок критичного шляху та оцінка продуктивності

Критичний шлях — це найдовший ланцюжок послідовних операцій від моменту зміни сигналу тактового генератора до повної стабілізації результату в кінцевій точці. У спрощеній моделі RISC-V критичним шляхом є виконання команди **LW (Load Word)**, оскільки вона проходить через найбільшу кількість послідовних вузлів.

Схема критичного шляху:

РС -> Пам'ять інструкцій -> Регістровий файл -> АЛП -> Пам'ять даних -> Мультиплексор -> Регістровий файл (вхід запису).

Розрахунок періоду такту (T_{cycle}):

Для стабільної роботи процесора період такту повинен бути більшим або дорівнювати сумі затримок усіх компонентів на критичному шляху:

$$T_{\text{cycle}} \geq t_{\text{pc}} + t_{\text{mem_inst}} + t_{\text{reg_read}} + t_{\text{alu}} + t_{\text{mem_data}} + t_{\text{mux}} + t_{\text{reg_setup}}$$

Припустимо наступні типові затримки для синтезованої логіки (у наносекундах):

- $t_{\text{pc}} = 1$ нс (час оновлення лічильника);
- $t_{\text{mem}} = 10$ нс (час доступу до пам'яті інструкцій та даних);
- $t_{\text{reg}} = 3$ нс (час зчитування з регістрового файлу);
- $t_{\text{alu}} = 5$ нс (час виконання операції в АЛП);
- $t_{\text{mux}} = 1$ нс (затримка на мультиплексорі);

$t_{reg_setup} = 2 \text{ нс}$ (час стабілізації перед записом).

Підстановка значень: $T_{cycle} = 1 + 10 + 3 + 5 + 10 + 1 + 2 = 32 \text{ нс}$.

Розрахунок максимальної частоти (f_{max}):

Максимальна тактова частота визначається як величина, обернена до періоду такту:

$$f_{max} = 1 / T_{cycle} = 1 / 32 \text{ нс} = 31.25 \text{ МГц}.$$

Висновок щодо продуктивності

Отримана частота у **31.25 МГц** є теоретичною межею швидкодії для даної однотактної архітектури без використання конвеєризації. Хоча цей показник значно нижчий за частоти сучасних процесорів, він є цілком достатнім для спеціалізованих вбудованих систем керування та демонстрації принципів роботи RISC-архітектур.

ВИСНОВКИ

У ході виконання курсової роботи було успішно спроектовано та досліджено спрощений тракт даних процесора на базі архітектури RISC-V. Розроблена модель охоплює основні типи інструкцій RV32I, що дозволяє виконувати повноцінні цілочисельні обчислення та логічне керування програмою. Виконане технічне завдання підтвердило можливість реалізації функціонального ядра CPU з мінімальним набором компонентів. Аналіз результатів показав, що однотактна архітектура має вагому перевагу у вигляді простоти логіки керування, проте її продуктивність обмежена найдовшою командою (LW). Розрахована максимальна частота 31.25 МГц є прийнятною для навчальних цілей, але недостатньою для високопродуктивних обчислень. Для майбутньої модернізації рекомендується впровадження конвеєрної обробки (Pipelining). Розбиття циклу виконання на 5 стадій дозволить значно скоротити критичний шлях і, як наслідок, підвищити тактову частоту процесора в кілька разів, зберігаючи при цьому ту саму систему команд.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

- Hennessy, J. L., & Patterson, D. A. (2017). Computer Architecture: A Quantitative Approach (6th ed.). Morgan Kaufmann.
- Patterson, D. A., & Waterman, A. (2017). The RISC-V Reader: An Open Architecture Atlas. Strawberry Canyon.
 - Waterman, A., Lee, Y., Patterson, D., & Asanović, K. (2021). The RISC-V Instruction Set Manual, Volume II: Privileged Architecture. UC Berkeley.
 - Мельник, А. О. (2008). Архітектура комп'ютера. Наукова думка.
 - Stallings, W. (2018). Computer Organization and Architecture: Designing for Performance (11th ed.). Pearson.
 - Жуков, І. А., & Корочкін, О. В. (2008). Архітектура комп'ютерних систем. Корнійчук.
 - Матвієнко, М. П., & Розен, В. П. (2013). Архітектура комп'ютерів. Ліра-К.
 - Asanović, K., & Patterson, D. A. (2014). Instruction Sets Should Be Free: The Case For RISC-V. Technical Report UCB/EECS-2014-146, UC Berkeley.
 - Бабич, М. П., & Жуков, І. А. (2004). Комп'ютерна схемотехніка. МК-Прес.
 - Null, L., & Lobur, J. (2018). The Essentials of Computer Organization and Architecture (5th ed.). Jones & Bartlett Learning